The integrated explicit analytic number theory network

Terence Tao · 15 January 2026

https://terrytao.wordpress.com/2026/01/15/the-integrated-explicit-analytic-number-theory-network/

tao2typ · converted to Typst 2026-10-14 (UTC)

Like many other areas of modern analysis, analytic number theory often relies on the convenient device of asymptotic notation to express its results. It is common to use notation such as $X = O(Y)$ or $X \ll Y$, for instance, to indicate a bound of the form $|X| \leq CY$ for some unspecified constant C. Such implied constants C vary from line to line, and in most papers, one does not bother to compute them explicitly. This makes the papers easier both to write and to read (for instance, one can use asymptotic notation to conceal a large number of lower order terms from view), and also means that minor numerical errors (for instance, forgetting a factor of two in an inequality) typically has no major impact on the final results. However, the price one pays for this is that many results in analytic number theory are only true in asymptotic sense; a typical example is Vinogradov's theorem that every sufficiently large odd integer can be expressed as the sum of three primes. In the first few proofs of this theorem, the threshold for "sufficiently large" was not made explicit.

There is however a small portion of the analytic number theory devoted to *explicit* analytic number theory estimates, in which all constants are made completely explicit (and many lower order terms are retained). For instance, whereas the prime number theorem asserts that the prime counting function $\pi(x)$ is asymptotic to the logarithmic integral $\mathrm{Li}(x) = \int_2^x \frac{dt}{\log t}$, in this recent paper of Fiori, Kadiri, and Swidinsky the explicit estimate

$$|\pi(x) - \mathrm{Li}(x)| \leq 9.2211 x \sqrt{\log x} \exp(-0.8476\sqrt{\log x})$$

is proven for all $x \geq 2$.

Such explicit results follow broadly similar strategies of proof to their non-explicit counterparts, but require a significant amount of careful book-keeping and numerical optimization; furthermore, any given explicit analytic number theory paper is likely to rely on the numerical results obtained in previous explicit analytic number theory papers. While the authors make their best efforts to avoid errors and build in some redundancy into their work, there have unfortunately been a few cases in which an explicit result stated in the published literature contained numerical errors that placed the numerical constants in several downstream applications of these papers into doubt.

Because of this propensity for error, updating any given explicit analytic number theory result to take into account computational improvements in other explicit results (such as zero-free regions for the Riemann zeta function) is not done lightly; such updates occur on the timescale of decades, and only by a small number of specialists in such careful computations. This leads to authors needing such explicit results to often be forced to rely on papers that are a decade or more out of date, with constants that they know in principle can be improved by inserting more recent explicit inputs, but do not have the domain expertise to confidently update all the numerical coefficients.

To me, this situation sounds like an appropriate application of modern AI and formalization tools – not to replace the most enjoyable aspects of human mathematical research, but rather to allow extremely tedious and time-consuming, but still necessary, mathematical tasks to be offloaded to semi-automated or fully automated tools.

Because of this, I (acting in my capacity as Director of Special Projects at IPAM) have just launched the integrated explicit analytic number theory network, a project partially hosted within the existing "Prime Number Theorem And More" (PNT+) formalization project. This project will consist of two

components. The first is a crowdsourced formalization project to formalize a number of inter-related explicit analytic number theory results in Lean, such as the explicit prime number theorem of Fiori, Kadiri, and Swidinsky mentioned above; already some smaller results have been largely formalized, and we are making good progress (especially with the aid of modern AI-powered autoformalization tools) on several of the larger papers. The second, which will be run at IPAM with the financial and technical support of Math Inc., will be to extract from this network of formalized results an interactive "spreadsheet" of a large number of types of such estimates, with the ability to add or remove estimates from the network and have the numerical impact of these changes automatically propagate to other estimates in the network, similar to how changing one cell in a spreadsheet will automatically update other cells that depend on it. For instance, one could increase or decrease the numerical threshold to which the Riemann hypothesis is verified, and see the impact of this change on the explicit error terms in the prime number theorem; or one could "roll back" all the literature to a given date, and see what the best estimates on various analytic number theory expressions could still be derived from the literature available at that date. Initially, this spreadsheet will be drawn from direct adaptations of the various arguments from papers formalized within the network, but in a more ambitious second stage of the project we plan to use AI tools to modify these arguments to find more efficient relationships between the various numerical parameters than were provided in the source literature.

These more ambitious outcomes will likely take several months before a working proof of concept can be demonstrated; but in the near term I will be grateful for any contributions to the formalization side of the project, which is being coordinated on the PNT+ Zulip channel and on the github repository. We are using a github issues based system to coordinate the project, similar to how it was done for the Equational Theories Project. Any volunteer can select one of the outstanding formalization tasks on the Github issues page and "claim" it as a task to work on, eventually submitting a pull request (PR) to the repository when proposing a solution (or to "disclaim" the task if for whatever reason you are unable to complete it). As with other large formalization projects, an informal "blueprint" is currently under construction that breaks up the proofs of the main results of several explicit analytic number theory papers into bite-sized lemmas and sublemmas, most of which can be formalized independently without requiring broader knowledge of the arguments from the paper that the lemma was taken from. (A graphical display of the current formalization status of this blueprint can be found here. At the current time of writing, many portions of the blueprint are disconnected from each other, but as the formalization progresses, more linkages should be created.)

One minor innovation implemented in this project is to label each task by a "size" (ranging from XS (extra small) to XL (extra large)) that is a subjective assessment of the task difficulty, with the tasks near the XS side of the spectrum particularly suitable for beginners to Lean.

We are permitting AI use in completing the proof formalization tasks, though we require the AI use to be disclosed, and that the code is edited by humans to remove excessive bloat. (We expect some of the AI-generated code to be rather inelegant; but *no* proof of these explicit analytic number theory estimates, whether human-generated or AI-generated, is likely to be all that pretty or worth reading for its own sake, so the downsides of using AI-generated proofs here are lower than in other use cases.) We of course require all submissions to typecheck correctly in Lean through Github's Continuous Integration (CI) system, so that any incorrect AI-generated code will be rejected. We are also cautiously experimenting with ways in which AI can also automatically or semi-automatically generate the formalized *statements* of lemmas and theorems, though here one has to be significantly more alert to the dangers of misformalizing an informally stated result, as this type of error cannot be automatically detected by a proof assistant.

We also welcome suggestions for additional papers or results in explicit analytic number theory to add to the network, and will have some blueprinting tasks in addition to the formalization tasks to convert such papers into a blueprinted sequence of small lemmas suitable for individual formalization.

Update: a personal log documenting the project may be found here.

Comments (8)

Anonymous · 16 January 2026

Terence Tao is a gift to humanity.

Anonymous · 16 January 2026

There's the initiative of Olivier Ramaré and colleagues which seems relevant: https://ramare-olivier.github.io/TME-EMT/accueil.html

> **Terence Tao** · 16 January 2026
>
> Yes, when we have optimized our workflows a bit more we certainly plan to input the results from the TME-EMT network into our own network (initially as "sorries" with no proofs provided, but eventually with their proofs formalized also). Right now we are focusing a relatively small number of papers to see if we can actually accomplish an end-to-end explicit prime number theorem with all the prerequisite results from previous papers also formalized, but once we have that infrastructure set up it should be relatively straightforward to add these results.
>
> > **André Pierro de Camargo** · 17 January 2026
> >
> > Allow me to add that we are currently transfering Oliver's project to a new site (see url below) to make updating and further mantainance easier. We shall create a wikipedia link to it when it is ready (there are still some typeseting and typos introduced during the transfer to be fixed).
> > The new webpage is: https://tme-emt-wiki-gitlab-io-9d3436.gitlab.io/
> >
> > > **Anonymous** · 30 April 2026
> > >
> > > Just an update: the new site is:
> > > TME-EMT-Wiki 01 documentation

secretlyobject05ab30fe9f · 20 January 2026

This project effectively moves the field from a 'static archive' model to a 'continuous integration' model.
By formalizing these explicit bounds, you are essentially treating a theorem not as a fixed artifact (a PDF), but as a function: $Theorem(Input_Bounds) to Output_Bounds$.
The 'spreadsheet' then acts as the runtime environment. The beauty of this approach is that it resolves the latency between a foundational improvement (e.g., a better zero-free region) and its downstream consequences. Instead of waiting decades for a new survey paper to 'clear the cache,' the network can just 'recompile' the dependent constants.
It suggests that eventually, the network itself becomes the canonical, dynamically maintained representation of the state of the art, with traditional papers serving as human-readable snapshots of particular states in the graph.

Anonymous · 23 January 2026

A major concern is this direction is how to handle the rounding errors introduced during the process. Say an explicit estimate E was obtained in a paper "A" by means of constants E_1, E_2, ... E_k published elsewhere. When E_1, E_2 ... E_k are updated, the new "E" shall be computed in finite precision and the rounding errors introduced in the computation of E must be taking into account. This analysis is usually done by the researcher which is involved in the updating process, and it is not trivial how to automatize this part. In fact, it seems that the unique automatic tool that ensures reliable bounds for the computed "E" is Interval arithmetic. However, in some cases, it leads to non-realistic (although correct) bounds.



Terence Tao · 23 January 2026

If the analysis is formalized in Lean, it will have had to deal with these roundoff errors; floating point arithmetic is certainly not recognized as a valid substitute for exact arithmetic in Lean! Therefore in the updating process, one may initially use floating point arithmetic to obtain an initial guess for what values the updated downstream constants should be, but then one has to actually formalize this within Lean, and some small adjustments to the constants may then be needed if the roundoffs do not work in one's favor. Based on some early AI experiments with my UCLA group we believe that current AI models are (barely) able to perform this type of adjustment, at least for relatively simple functional relationships between constants – and if the constants are not adjusted so drastically that the global shape of the argument has to change as a consequence – but we need to work to see whether it can be scaled up to the complex arguments that will appear in this network.

However, there will inevitably be some very large computations in this network that will not be feasible to formalize in Lean – numerical verification of the Riemann Hypothesis up to a very large height H being the prime example. This will probably have to be represented within Lean as an axiom, and in many cases the only known verification of such axioms will be through nonrigorous arithmetic, such as floating point arithmetic. However, my hope is that it will be relatively easy (particularly with AI tools) to tune down this height parameters H and recalculate all of the downstream constants in a short amount of time (and in a Lean-verified fashion), so that one can see just how much numerical verification is actually needed to establish any given result, and in particular whether one can retreat to a numerical RH result which is weaker, but more rigorously verified (e.g., by an interval arithmetic calculation in a conventional software language such as Python).